

ЛЕКЦИЯ 4 · ИНСТРУМЕНТЫ РАЗРАБОТКИ

Bash-скрипты и отладка сервера

Как автоматизировать рутину скриптами, писать их надёжно и диагностировать сервер, когда что-то пошло не так

Чернов Евгений
Магистратура «Искусственный интеллект», НИУ ВШЭ

О ЧЁМ ЛЕКЦИЯ

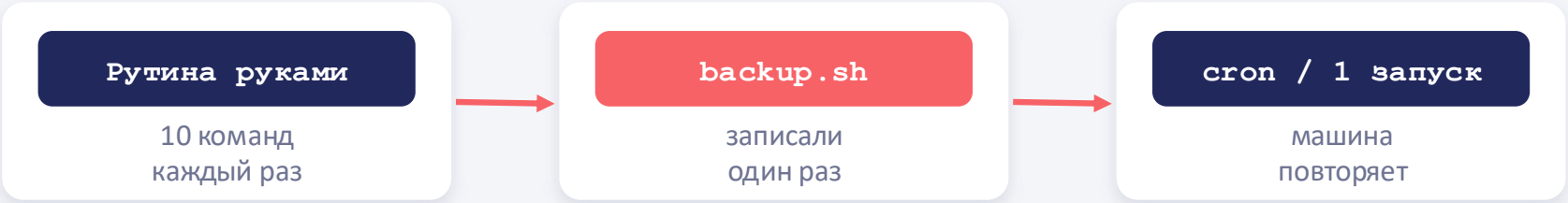
План занятия

1. **Зачем скрипты**
   автоматизация рутины и повторяемость
2. **Анатомия скрипта**
   shebang, переменные, кавычки, exit codes
3. **Логика**
   if / for / while / case, функции, аргументы
4. **Надёжность**
   set -euo pipefail, trap, запуск по cron
5. **Отладка**
   bash -x, set -x, точечный echo
6. **Диагностика сервера**
   CPU, память, диск, порты, сервисы, логи

ПОСТАНОВКА ЗАДАЧИ

Зачем нужны скрипты?

Скрипт — это последовательность команд, записанная в файл. Пишете один раз — выполняете сколько угодно раз: одинаково, быстро и без забытых шагов.



Правило: всё, что делаете руками больше двух раз, — заверните в скрипт. Компьютер не устаёт и не забывает шаг.

ОСНОВЫ

Анатомия скрипта

hello.sh

```
#!/usr/bin/env bash
name="Иван"
echo "Привет, $name!"
echo 'литерал: $name'
n=$(ls | wc -l)
echo "файлов: $n"
exit 0
```

#! shebang

какой интерпретатор запускает файл

var=...

переменные; без пробелов вокруг =

"$var"

двойные кавычки подставляют, одинарные — нет

exit 0 · $?

код выхода: 0 — успех, ≠0 — ошибка

ОСНОВЫ

Как создать и запустить скрипт

терминал

```
$ nano hello.sh
$ ./hello.sh
bash: ./hello.sh: Permission denied
$ chmod +x hello.sh
$ ./hello.sh
Привет, Мир!
$ bash hello.sh
Привет, Мир!
```

chmod +x

разрешить запускать файл как программу

./hello.sh

запуск из текущей папки — нужна «./»

bash hello.sh

запуск явным bash — chmod не нужен

Permission denied

забыли chmod +x — частая первая ошибка

ОСНОВЫ

Кавычки и подстановка команд

кавычки решают всё

```
name="Аня"
echo "привет, $name"
привет, Аня
echo 'привет, $name'
привет, $name
now=$(date +%H:%M)
echo "сейчас ${now}"
сейчас 14:03
```

"$var"
двойные кавычки — подставляют значение

'$var'
одинарные — берут текст буквально

${var}
фигурные скобки — граница имени

$(cmd)
подставить вывод команды в строку

ЛОГИКА

Управляющие конструкции

if / for

```
if [ -f "$f" ]; then
  echo "есть файл"
else
  echo "нет файла"
fi

for i in 1 2 3; do
  echo "шаг $i"
done
```

while / case

```
n=0
while [ $n -lt 3 ]; do
  n=$((n + 1))
done
case "$1" in
  start) run ;;
  stop)  halt ;;
  *) usage ;;
esac
```

ЛОГИКА

Проверки: [ ], [[ ]] и сравнения

Квадратные скобки […] — это команда-проверка test. Она возвращает код 0 (истина) или 1 (ложь), а if по нему ветвится.

test / […]

```
[ -f config.txt ]    # файл существует?
[ -d /var/log ]      # каталог есть?
[ -z "$x" ]          # строка пустая?
[ "$n" -eq 10 ]      # числа: -eq -lt -gt
[ "$s" = "yes" ]     # строки:  =  !=
[[ "$s" == y* ]]     # шаблоны и &&
```

-f -d -e

файл / каталог / существует

-z -n

строка пустая / непустая

-eq -lt -gt

сравнение ЧИСЕЛ

= !=

сравнение СТРОК — не -eq!

ЛОГИКА

Коды выхода: контракт скрипта

Каждая команда возвращает код выхода: 0 — успех, любое другое число — ошибка. На этом строится вся логика скриптов.

код выхода $?

```
$ grep root /etc/passwd; echo $?
root:x:0:0:root:/root:/bin/bash
0
$ grep zzz /etc/passwd; echo $?
1

mkdir d && cd d       # cd, если mkdir ок
cp a b || echo fail   # echo при ошибке
```

$?
код выхода последней команды

exit 0 / 1
чем скрипт отвечает системе

cmd && ok
выполнить, только если успех

cmd || fix
выполнить, только если ошибка

ЛОГИКА

Функции, аргументы и ввод

args.sh

```
greet() {
  echo "Привет, $1!"
}
greet "Иван"

echo "скрипт:    $0"
echo "аргументов: $#"
echo "все:       $@"
read -p "Имя? " user
```

`$0`	имя запущенного скрипта
`$1 $2`	позиционные аргументы
`$#`	сколько передано аргументов
`$@`	все аргументы списком
`$?`	код выхода прошлой команды
`read`	прочитать ввод пользователя

НАДЁЖНОСТЬ

Скрипт не должен падать молча

По умолчанию bash игнорирует ошибки и идёт дальше. Три флага и trap делают поведение предсказуемым, а очистку — гарантированной.

safe.sh

```
#!/usr/bin/env bash
set -euo pipefail

tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT

curl -fsS "$URL" > "$tmp"
process "$tmp"
```

set -e
выйти при первой же ошибке команды

set -u
падать на необъявленной переменной

pipefail
ловить ошибку в середине пайпа |

trap
выполнить код при выходе и сигналах

РАЗБОР ПРИМЕРА

Разбор: backup.sh построчно

backup.sh

```
#!/usr/bin/env bash
set -euo pipefail

src="$1"                  # папка-источник
dst="/tmp/backup"

if [ -z "$src" ]; then    # аргумент задан?
  echo "usage: backup.sh <dir>"
  exit 1
fi

mkdir -p "$dst"
for f in "$src"/*.txt; do
  cp "$f" "$dst"
done
```

`set -euo pipefail`
строгий режим — падаем на первой ошибке

`проверка $1`
нет аргумента → usage и exit 1

`for + cp`
копируем каждый .txt по одному

`"$src"  "$f"`
кавычки — пути с пробелами не рвутся

АВТОМАТИЗАЦИЯ

cron — запуск по расписанию

cron — планировщик: запускает команды по времени. Строка crontab — это 5 полей времени плюс сама команда.



! `0 3 * * 1 backup.sh` → каждый понедельник в 03:00 запустить бэкап.

ЧАСТЫЕ ОШИБКИ

Частые грабли bash

`name = 5`

пробелы вокруг = → bash ищет команду name. Верно: name=5

`rm $file`

пробел в имени рвёт аргумент. Всегда пишите "$file"

`[ $n == 5 ]`

== это для строк; числа сравнивают -eq: ["$n" -eq 5]

`bad interpreter`

Windows-переносы строк (CRLF). Лечит dos2unix file.sh

`Permission denied`

забыли chmod +x — или запускайте bash file.sh

`в cron не работает`

у cron пустой $PATH — пишите полные пути к командам

01

РАЗДЕЛ

Отладка и эксплуатация сервера

Находим, почему скрипт или сервер ведут себя не так, — и чиним по горячим следам

ОТЛАДКА

Отладка скрипта: видеть каждый шаг

Не гадайте, почему скрипт падает, — включите трассировку. Bash напечатает каждую команду после подстановок, прямо перед выполнением.

трассировка

```
$ bash -x backup.sh
+ src=/data
+ dst=/backup
+ tar -czf /backup/db.tgz /data
+ echo 'бэкап готов'
бэкап готов
+ exit 0
```

`bash -x file.sh`

трассировка без правки самого файла

`set -x` · `set +x`

включить / выключить трассировку в участке

`PS4="+ "`

префикс строк трассировки (можно менять)

`echo` · `printf`

точечно вывести значение переменной

ДИАГНОСТИКА

Нагрузка: CPU и память (top / htop)

Сервер тормозит — первым делом смотрят top: кто ест процессор и память, какова средняя нагрузка.

снимок нагрузки

```
$ top
load average: 4.51, 3.98, 2.10
%Cpu: 91.2 us, 5.1 sy, 3.7 id
MiB Mem: 7861 total, 210 free
  PID USER %CPU %MEM COMMAND
 3120 www  88.5 12.4 train.py
 2044 pg   10.2 30.1 postgres
```

load average

очередь к CPU; > числа ядер — перегрузка

%Cpu us/sy/id

юзер / система / простой процессора

%MEM · RES

сколько памяти реально ест процесс

htop

то же, но цветное, с деревом и мышью

ДИАГНОСТИКА

Диск (df / du) и порты (ss)

Две частые причины «сервер лёг»: закончилось место на диске или нужный порт никто не слушает.

диск

```
$ df -h /
Size Used Avail Use%
 40G  38G  0.5G  99%
$ du -sh /var/log/*
1.9G /var/log/journal
850M /var/log/nginx
```

порты

```
$ ss -tulpn
State  Port   Process
LISTEN *:80    nginx
LISTEN *:5432  postgres
LISTEN *:22    sshd
# порт 80 слушает nginx
```

df — место по разделам, **du** — что занимает папку, **ss** — кто слушает порт (замена netstat).

ДИАГНОСТИКА

Сервисы и логи: systemctl / journalctl

Фоновые программы живут как systemd-сервисы. Их состоянием управляют systemctl, а логи читают journalctl.

терминал

```
$ systemctl status nginx
● nginx - active (running)
$ systemctl restart nginx
$ journalctl -u nginx -n 50
$ journalctl -f
14:03 nginx: started
$ journalctl -p err -b
```

`status`	состояние: active / failed
`restart`	перезапуск (start/stop/enable)
`-u nginx`	логи одного сервиса
`-f`	следить в реальном времени
`-p err -b`	ошибки с последней загрузки

ДИАГНОСТИКА

Где живут логи и как их читать

Логи — дневник системы. Текстовые лежат в /var/log, у systemd-сервисов журнал бинарный — его читают через journalctl.

чтение логов

```
$ tail -n 50 /var/log/syslog
$ tail -f /var/log/nginx/error.log
  # -f: новые строки онлайн
$ grep -i error /var/log/syslog
$ less /var/log/syslog   # листать, q
$ journalctl -u nginx -p err -b
```

/var/log/
текстовые логи системы и программ

tail -f
новые строки в реальном времени

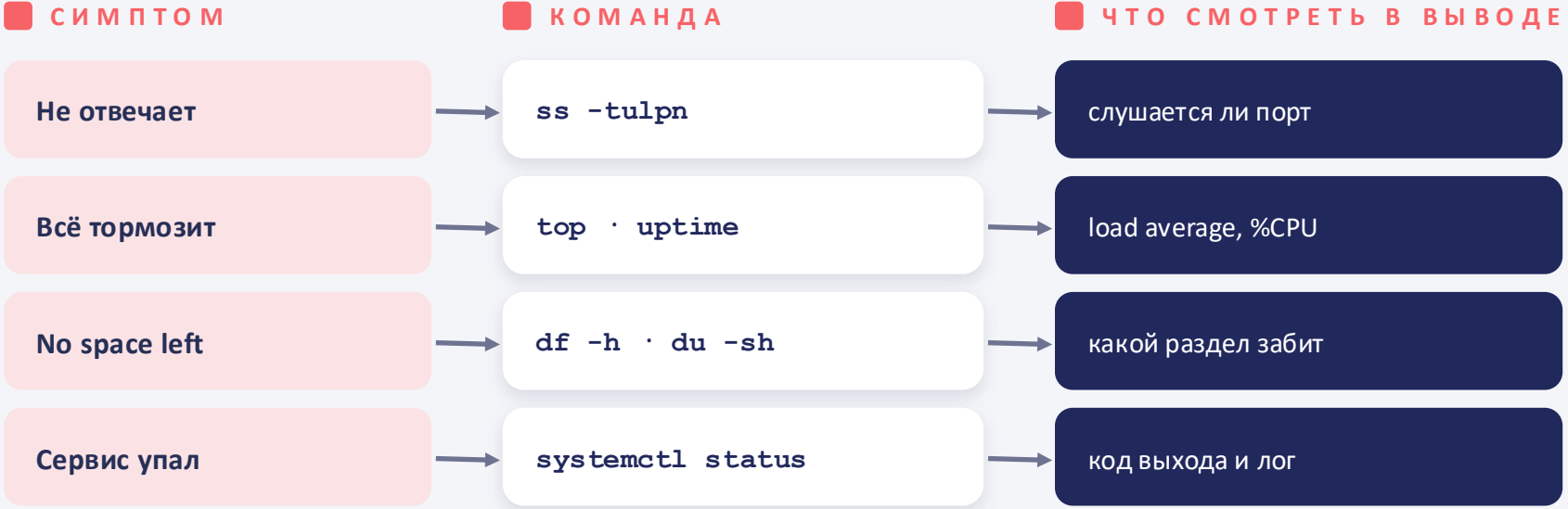
grep -i
фильтр строк по слову (error, fail)

journalctl
логи systemd — где нет текстовых

ДИАГНОСТИКА

Дерево диагностики «сервер лёг»

Идите по дереву «симптом → команда → что искать в выводе» сверху вниз, пока не найдёте причину.



РАЗБОР ПРИМЕРА

Разбор: health-check.sh построчно

health-check.sh

```
#!/usr/bin/env bash
set -euo pipefail

# 1. заполнен ли диск?
use=$(df / | awk 'NR==2{print $5}' | tr -d '%')
if [ "$use" -gt 90 ]; then
  echo "ALERT: диск занят ${use}%"
fi

# 2. жив ли сервис?
if ! systemctl is-active --quiet nginx; then
  systemctl restart nginx
fi
```

df | awk | tr

берём Use% и убираем знак процента

[-gt 90]

число сравниваем -gt; >90% → тревога

is-active

жив ли сервис (--quiet — тихо)

if ! …

«!» инвертирует: не активен → рестарт

ДИСЦИПЛИНА

Хорошие практики bash-скриптов

`set -euo pipefail`

первой строкой — чтобы скрипт падал на ошибке, а не тихо ломался

Кавычки всегда

"$var" и "$@" — иначе пробелы и пустые значения всё сломают

Проверяйте код

if ! cmd; then ...; проверяйте $? важных шагов

Логируйте ход

echo / logger с датой — потом поймёте, что и когда шло

Идемпотентность

повторный запуск не должен ломать уже сделанное

shellcheck

линтер ловит типичные ошибки до запуска в бою

ИТОГИ ЛЕКЦИИ

Что важно унести

1. **Скрипт = автоматизация**
   записали рутину один раз — машина повторяет её без ошибок

2. **Пишите надёжно**
   set -euo pipefail, кавычки вокруг переменных, trap для очистки

3. **cron по расписанию**
   регулярные задачи запускаются сами, без человека

4. **Диагностика по дереву**
   симптом → команда (top/df/ss/journalctl) → вывод

Дальше — **контейнеры (Docker): упаковываем приложение и окружение целиком**, чтобы «работает у меня» стало «работает везде».